From Vector Autoregressions to AI-based Time Series Forecasting: A Review

Likai Chen* Weining Wang†

July 17, 2026

arXiv:2607.14279v1 [econ.EM] 15 Jul 2026

Abstract

Forecasting is a central goal of time-series analysis. This review centers on three major developments in recent AI-based time-series forecasting: transformers, large pretrained models for zero-shot forecasting, and diffusion-based generative forecasters. We connect these methods to the econometric tradition built around the vector autoregression (VAR) through a common object: the conditional distribution of the future given the past. The review is organized around three long-standing challenges: *high dimensionality*, *nonstationarity*, and *nonlinearity*. We argue that modern methods make progress by expanding the classical forecasting template: they allow more flexible dynamics, use larger information sets and training corpora, and represent richer predictive distributions. Yet they often lack the inferential and structural tools that make classical models useful for testing, explanation, and policy analysis. We close by outlining open problems where econometric tools remain important.

Keywords: vector autoregression; transformers; foundation models; denoising diffusion; probabilistic forecasting; high-dimensional time series.

1. Introduction

Forecasting is central to applied econometrics, with uses ranging from monetary policy analysis and risk management to demand planning and nowcasting. For four decades, the vector autoregression (VAR) of Sims [52] has provided a coherent way to study how multiple time series move together. Its appeal comes not only from forecasting performance, but also from the interpretive apparatus built around it: impulse-response analysis, Granger causality [17], and the cointegration and error-correction framework of Engle and Granger [13] and Johansen [23]. Standard graduate treatments [20, 39] still organize the field around this core. For many problems, especially small systems and settings where interpretation, valid inference, or long-run relationships matter, the VAR remains the natural starting point.

At the same time, methods imported from machine learning have expanded what is possible at scale. The transformer architecture [60] is a leading example. For time-series forecasting, it can be read as a nonlinear generalization of the VAR: fixed lag coefficients are replaced by data-dependent attention weights. In its standard forecasting form, however, it usually produces point forecasts or marginal quantiles rather than a full econometric model. Foundation models [3, 11, 63] push this logic further by pretraining once on a large and diverse collection of time series, then forecasting new series without task-specific estimation. Denoising diffusion models [21, 46] add a generative layer, sampling future paths from the predictive distribution and thereby representing forecast uncertainty more directly. Although these developments are often presented as a break from classical statistics, we argue that they are better understood as new approaches to the same forecasting problem.

*Washington University in St. Louis. likai.chen@wustl.edu.
†University of Bristol. weining.wang@bristol.ac.uk.

Our aim is not simply to translate between two literatures. The central point is an asymmetry: modern methods often forecast well, but offer less support for explanation. They do not naturally produce impulse responses, testable restrictions, long-run equilibria, or classical sampling uncertainty. Identification, valid inference, structural interpretation, and nonstationarity therefore remain open problems. We see these gaps not only as weaknesses, but also as opportunities. They are precisely the kinds of problems econometrics is designed to address. Tools such as the bootstrap, conformal prediction, cointegration, and change-point methods may help connect modern predictive models with econometric interpretation. The review is written with this goal in mind: each section places the methods against three recurring challenges, and the conclusion turns the remaining gaps into a research agenda.

As a running example, consider a small macroeconomic panel containing inflation, output growth, and a short-term interest rate. A classical VAR uses their joint history to forecast future values. An identified Structural VAR asks how a monetary policy shock propagates through inflation and output. A Vector Error Correction Model (VECM) is appropriate when the variables share a long-run equilibrium. A transformer replaces fixed lag coefficients with data-dependent weights. A diffusion forecaster describes uncertainty by sampling possible future macroeconomic paths. We return to this example throughout to distinguish forecasting performance from the econometric questions of identification, inference, and nonstationarity.

A unifying view. Every model in this review can be read as an answer to a single question: what is the conditional law of the future $\boldsymbol{y}_{t+1:t+H}$ given the past $\boldsymbol{y}_{1:t}$ and any covariates? A Gaussian VAR answers with fixed linear coefficients and a Gaussian predictive density. A transformer answers with data-dependent nonlinear attention weights, but typically returns only point or quantile summaries. A diffusion model answers by learning to sample from the predictive law without explicitly parameterising its density. The differences among these methods are differences in flexibility: they represent the same object, the conditional distribution, in different ways. Holding this object fixed clarifies both what is gained as we move from VARs to attention and diffusion, and what is lost: closed-form inference, orthogonalised shocks, and testable restrictions.

Three challenges. The classical linear Gaussian model makes three assumptions that real data often violate. Much of the history of time series analysis, classical and modern alike, can be read as the progressive relaxation of these assumptions.

1. **High dimensionality.** A VAR(L) on p series has $O(p^2L)$ parameters, so unrestricted estimation quickly becomes unreliable as p grows. Classical responses include factor structure, shrinkage, and sparsity through Lasso-type penalization. Each imposes an explicit restriction before estimation. Modern architectures address the same problem through channel independence, cross-variable attention, and pretraining on large collections of series from many domains. Their distinctive feature is adaptivity: the restriction is less often specified in advance and more often learned from data. In this sense, the dimensionality problem is partly relocated from task-specific estimation to pooled representation learning. A full theoretical account of when this adaptivity succeeds, and when it fails, is still missing.

2. **Nonstationarity.** Means, variances, and dependence structures drift over time, and series may contain unit roots, share long-run equilibria, or move across regimes. Classical time-series analysis matches these forms of instability with specific tools: unit-root testing, differencing, and cointegration for stochastic trends, change-point methods for abrupt breaks, and time-varying coefficients for gradual drift. The modern forecasting literature treats nonstationarity more indirectly, relying mainly on instance normalization, scaling, and conditioning on recent history.

3. **Nonlinearity.** Linear fixed-coefficient dynamics cannot capture regime-dependent propa-

gation, asymmetric responses, or non-Gaussian predictive shapes. Machine-learning models can instead be viewed as nonparametric methods for learning nonlinear dependence. Attention and diffusion learn nonlinear and non-Gaussian conditional laws without imposing a parametric form in advance. Attention is especially distinctive because it can compare tokens at arbitrary distances: classical models typically discount the remote past by construction, while attention weights past dates according to learned relevance. Distance and dependence are therefore decoupled.

These three challenges organize the rest of the review. Section 2 reviews the classical foundations and locates the three challenges within them. Section 3 develops the transformer as a data-dependent generalization of the vector autoregression, and especially Subsection 3.3 focuses on the representational question inherited from language modelling: what counts as a token of a time series? Section 4 reviews foundation models and zero-shot forecasting, emphasizing the shift from per-data-set estimation to cross-corpus pretraining. Section 5 reviews diffusion-based generative forecasters, emphasizing the move from marginal summaries to sampled joint forecast paths. Section 6 offers a synthesis and summary. Throughout, we aim less at exhaustive coverage than at a coherent narrative connecting modern forecasting methods to econometric concerns. Recent surveys provide broader coverage of transformer methods [61] and diffusion models [41, 67].

Notation and terminology. Throughout, $\boldsymbol{y}_t \in \mathbb{R}^p$ denotes the vector of p series observed at date t, and T denotes the sample length. The symbol L denotes the length of the look-back window. In Section 2, this is the lag order of the vector autoregression. From Section 3 onward, it is the context length of the sequence model. In both cases, the input is a window of the L most recent observations. The symbol H denotes the forecast horizon, so that $\boldsymbol{y}_{t+1:t+H}$ is the object to be predicted, and h indexes individual forecast steps.

2. Classical multivariate foundations

For a $p \times 1$ vector $\boldsymbol{y}_t$, the vector autoregression of order L is

$$\boldsymbol{y}_t = \boldsymbol{\nu} + \sum_{i=1}^{L} A_i \boldsymbol{y}_{t-i} + \boldsymbol{\varepsilon}_t, \qquad \boldsymbol{\varepsilon}_t \sim \text{iid}\,(\mathbf{0}, W), \tag{1}$$

where $A_i \in \mathbb{R}^{p\times p}$ are coefficient matrices and W is the innovation covariance matrix, which is generally non-diagonal. Because every equation shares the same regressors, the system is a seemingly unrelated regression in which equation-by-equation ordinary least squares is efficient, and a complete distribution theory is available [20, 39]. When the roots of $\det(I_p - A_1 z - \cdots - A_L z^L)$ lie outside the unit circle, the process is stable and admits the Wold representation

$$\boldsymbol{y}_t = \boldsymbol{\mu} + \sum_{h\geq 0} B_h \boldsymbol{\varepsilon}_{t-h},$$

with $B_0 = I_p$. The matrices B_h are the basis of structural analysis. Augmenting (1) with moving-average terms gives the vector autoregressive moving-average (VARMA) class. Under the same stability condition, a VARMA model also admits an infinite moving-average representation. It can therefore represent current outcomes as weighted accumulations of past shocks, often more parsimoniously than a finite VAR, although estimation and identification are harder [39]. This shock-accumulation form provides a useful, though only formal, parallel with the accumulation of artificial noise in diffusion models (Section 5).

In the macroeconomic panel example, $\boldsymbol{y}_t$ might collect inflation, output growth, and the policy rate. The coefficient matrix A_i records how the ith lag of each variable enters each equation. Lagged inflation may help forecast the policy rate, lagged policy rates may help forecast output

growth, and so on. This is a reduced-form predictive object. It becomes a structural object only after additional identifying assumptions are imposed.

Beyond producing forecasts, a fitted VAR is used to answer two further questions, both directly related to what modern methods do and do not deliver. The first is *propagation*: what happens after an innovation or shock enters the system? This leads to impulse-response analysis. The second is *predictive relevance*: does the history of one series improve forecasts of another? This leads to Granger-causality testing.

For propagation, the Wold representation gives the basic reduced-form object. The (i, j) entry of B_h records the response of variable i, h periods after a one-unit innovation to variable j. Plotting these entries against the horizon h gives an impulse-response function. A complication arises because W is generally not diagonal. The reduced-form errors are correlated across equations, so an innovation to one variable alone is not yet a well-defined structural experiment. To interpret an impulse response as the effect of an economically meaningful shock, such as a monetary policy shock, the correlated innovations must first be transformed into orthogonal structural shocks. This is the identification problem. Standard solutions impose additional assumptions, such as a recursive ordering of contemporaneous effects or restrictions on the signs or long-run impacts of particular shocks [8, 25].

In the running macroeconomic example, an impulse response might trace the effect of a monetary policy shock on output growth and inflation over subsequent quarters. The important point is that the shock is not simply an innovation in the interest-rate equation. Because reduced-form VAR innovations are correlated, the monetary policy shock must be identified by restrictions that give it structural meaning.

For predictive relevance, Granger [17] turned the question into a testable hypothesis. Let $y_{k,t}$ denote the kth component of $\boldsymbol{y}_t$. In the equation for $y_{k,t}$, the statement that $y_{\ell,t}$ fails to Granger-cause $y_{k,t}$ means that all coefficients on the lags of $y_{\ell,t}$ are zero. This is a restriction on the matrices $A_1, \ldots, A_L$, and it can be tested by a standard F or Wald statistic. For the same panel, the statement that the policy rate Granger-causes inflation means only that lagged policy rates improve forecasts of inflation, conditional on the other included lags. It does not mean that changing the policy rate would have the same effect as a policy intervention. This distinction between predictive content and structural causation must be preserved when interpreting modern forecasting models.

Impulse responses and Granger tests share a common requirement: fixed coefficients. The impulse responses B_h are functions of the coefficient matrices A_i, and the Granger test is a hypothesis about whether particular entries of those matrices are zero. If the coefficients vary across time or across input windows, there is no single impulse-response function to compute and no fixed coefficient on which to impose a zero restriction. This is exactly the constancy that self-attention discards. Fixed coefficients are replaced by weights recomputed from each input window. Figure 1 in Section 3 illustrates the contrast. Modern methods therefore gain forecasting flexibility, but lose the fixed objects that make explanation and testing straightforward.

Nonstationarity is treated directly in classical time-series analysis. The classical toolkit matches the type of instability to the model. For stochastic trends, many economic series are integrated of order one. Regressions among unrelated integrated series may therefore be spurious [18]. Unit-root tests [12, 45] help diagnose this problem. The constructive response is cointegration: $I(1)$ series may share a stationary long-run combination, in which case the corresponding representation is the vector error-correction form

$$\Delta \boldsymbol{y}_t = \boldsymbol{\alpha}\, \boldsymbol{\beta}^\top \boldsymbol{y}_{t-1} + \sum_{j=1}^{L-1} \Gamma_j\, \Delta \boldsymbol{y}_{t-j} + \boldsymbol{\varepsilon}_t, \tag{2}$$

where Δ denotes the first difference, $\boldsymbol{\beta} \in \mathbb{R}^{p \times r}$ collects the r cointegrating vectors defining the stationary combinations $\boldsymbol{\beta}^\top \boldsymbol{y}_t$, $\boldsymbol{\alpha} \in \mathbb{R}^{p \times r}$ gives the speeds at which deviations from those combinations are corrected, and the Γ_j are short-run dynamic coefficients. The cointegrating

rank r and the coefficient matrices can be estimated by the reduced-rank procedure of Johansen [23, 24], building on Engle and Granger [13] and the unit-root distribution theory of Dickey and Fuller [12], Phillips and Perron [45].

Abrupt instability is handled differently. Structural change becomes an estimation problem: the number and locations of change points are parameters of the process. Segmentation estimators can be consistent with near-optimal localization rates even when the number of breaks grows with the sample; wild binary segmentation is one example [15]. More recently, neural networks have also been trained as change-point detectors with accompanying theory [for example 30]. For gradual instability, parameters may instead be allowed to drift smoothly. This is the idea behind the locally stationary framework of Dahlhaus [10], where the process is approximately stationary within short windows and can be estimated by kernel or local-likelihood methods. Time-varying parameter models in econometrics follow the same logic.

The practical lesson recurs below. If the data are cointegrated, a model that uses differences alone is misspecified because it discards the equilibrium-restoring term in (2). Neural and generative forecasters can effectively make this move when long-run structure is not built into the model. In the macroeconomic example, cointegration would matter if some variables shared a stable long-run relation, so that deviations from that relation predict later adjustment. A model estimated only in differences may forecast short-run movements well while discarding the term that gives the system its long-run discipline.

The three challenges. High dimensionality is already binding in the VAR, since the unrestricted system (1) contains $O(p^2L)$ parameters. Classical remedies are explicit and interpretable: shrinkage through Bayesian priors of the Minnesota family [6, 34]; dimension reduction through dynamic factor models [5, 58]; and, more recently, sparsity through regularised high-dimensional VARs [7, 27]. Each makes estimation feasible by imposing a stated restriction, in contrast to the implicit, data-driven regularization of the methods discussed below.

Nonstationarity, as we have just seen, is handled by tools matched to the type of instability: cointegration for stochastic trends, change-point estimation for abrupt breaks, and smoothly varying coefficients for gradual drift.

Nonlinearity remains the main limitation. The VAR in (1) is linear and has constant coefficients, so regime-dependent propagation, asymmetric responses, and non-Gaussian predictive shapes lie outside its scope. High dimensionality and nonlinearity are natural strengths of modern machine-learning methods, which can scale to large panels and learn flexible dependence structures. Nonstationarity remains the axis on which these methods have the most to learn from econometrics, with its established tools for unit roots, cointegration, structural breaks, and parameter drift.

3. The transformer era

Many nonlinear architectures have been brought to time-series prediction, including recurrent networks, temporal convolutions, and deep basis-expansion models. These methods relax the linearity of (1) by replacing fixed linear dynamics with nonlinear function approximators. Earlier global models had already moved beyond per-series estimation. DeepAR [48] used a recurrent network to produce an autoregressive predictive distribution. N-BEATS [44] used stacked residual blocks with trend and seasonal basis expansions, and N-HiTS [9] extended this idea with multi-rate pooling and hierarchical interpolation.

This section focuses on the transformer, now the central architecture in neural forecasting. Its importance does not lie simply in introducing nonlinearity. That shift had already begun. The transformer era is distinctive because it makes attention and tokenization the organizing devices for neural forecasting. We first introduce the basic attention mechanism and show how it replaces the fixed lag coefficients of a VAR with weights that depend on the input window. We

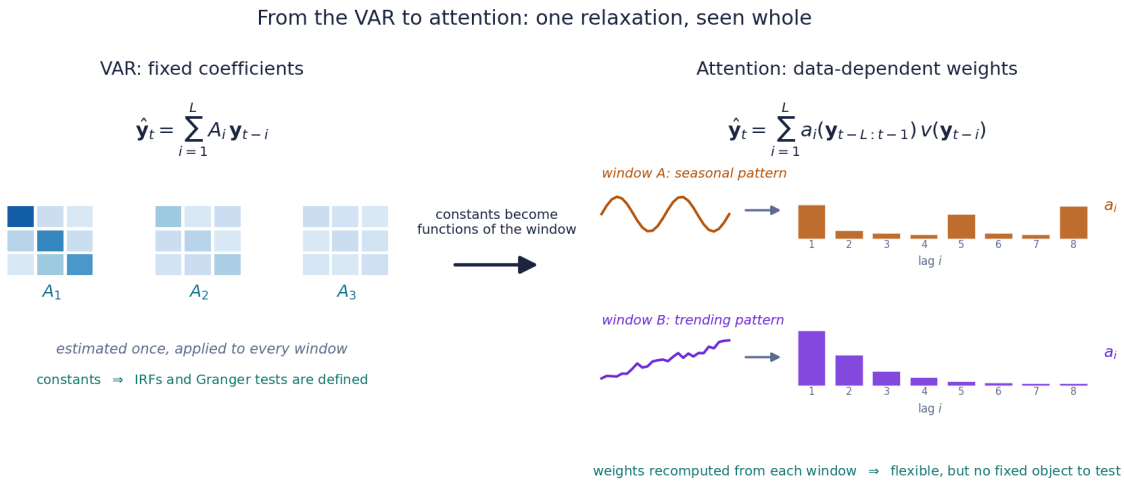


Figure 1: The central relaxation of this review. Left: the VAR forecasts with fixed coefficient matrices A_i, estimated once and applied to every window. This constancy is what defines impulse responses and Granger tests. Right: attention forecasts with the same summation structure but with weights $a_i(\cdot)$ that are functions of the input window and are recomputed at every date. A seasonal window and a trending window can therefore induce different lag profiles from the same fitted model. The gain is adaptivity and nonlinearity. The loss is the fixed object on which the classical inferential apparatus rests. Equation (5) states the correspondence precisely.

then discuss the forecasting-specific pressures that shaped early time-series transformers: long look-back windows, multi-step prediction, seasonality, distribution shift, and nonstationarity. The final part of the section treats tokenization as a central design choice, covering point-wise tokens, patch-based models, and variable-token models. The probabilistic dimension returns in Section 5, where we discuss generative models.

3.1 The basic transformer mechanism

The transformer of Vaswani et al. [60] replaces recurrence with self-attention. For a time-series reader, the central object is the attention weight matrix. It plays a role analogous to lag weights in an autoregression, but these weights are recomputed from the current input window. Figure 1 previews this comparison. The summation structure of the VAR is retained, while fixed coefficient matrices are replaced by input-dependent weights.

Let the input be a sequence of L token vectors collected in the rows of $X \in \mathbb{R}^{L\times d}$. Rows index token positions, while columns index embedding coordinates. The meaning of a "token" in time series is deferred to Subsection 3.3. Attention forms queries, keys, and values through learned projections,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K \in \mathbb{R}^{d\times d_k}$ and $W_V \in \mathbb{R}^{d\times d_v}$. It then returns

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{3}$$

The softmax is applied to each row of its matrix argument. For a vector $z = (z_1,\dots,z_L)^\top$, it is defined by

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{L}\exp(z_j)}, \qquad i = 1,\dots,L. \tag{4}$$

Each row of the resulting weight matrix is therefore nonnegative and sums to one. Each output row of (3) is consequently a convex combination of the rows of V.

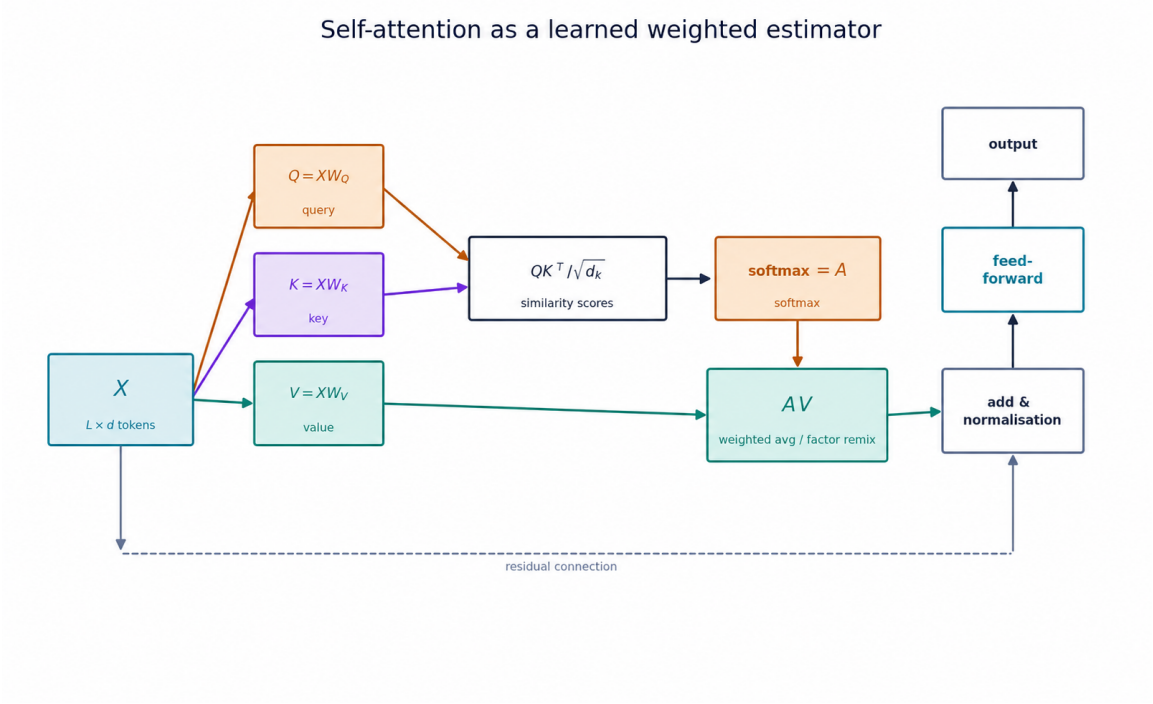


Figure 2: Self-attention as a learned weighted estimator. The input X is projected to queries, keys, and values; the scaled scores $QK^\top/\sqrt{d_k}$ pass through a row-wise softmax to form the weight matrix A, and the output AV is a weighted average of the values, followed by a residual connection, normalization, and a position-wise feed-forward map, over M heads. Two econometric readings are available: A collects Nadaraya–Watson kernel weights under a learned kernel, and equivalently the columns of V act as learned common factors with the rows of A as data-dependent loadings.

Three properties are worth isolating. First, the row weights in (3) are data dependent. Unlike fixed regression coefficients, they are recomputed for each input from that input's own content. Second, the map is global. Since rows of X index token positions, any two rows can interact directly through the $L \times L$ score matrix $QK^\top$. Dependence between distant token positions therefore need not pass through intermediate states. Third, the normalization by $\sqrt{d_k}$ is substantive. When the entries of Q and K are approximately independent with unit variance, the inner products $q^\top k$ have variance of order d_k. Without this scaling, the softmax can put almost all weight on a single key, making the attention weights hard to adjust during training.

A complete transformer block applies (3) in M parallel heads, typically with head dimension d/M. The head outputs are concatenated and mixed by an output projection. A position-wise feed-forward network, residual connections, and layer normalization complete the standard block. Because attention itself is permutation equivariant, ordering is supplied separately through positional encodings. For forecasting, a causal mask sets the scores for future positions to $-\infty$. After the softmax, those entries receive zero weight, so each position can attend only to itself and earlier positions. Figure 2 summarizes the standard transformer block.

3.2 Adapting transformers to forecasting

The connection to (1) is exact in form. To make the comparison explicit, let E denote the embedding map that turns an observed vector into a token vector. For a one-step forecast at date t, write

$$x_s = E(\boldsymbol{y}_s), \qquad X_t = [x_{t-L}, x_{t-L+1}, \ldots, x_{t-1}]^\top \in \mathbb{R}^{L\times d}.$$

Thus X_t is the embedded look-back window. Raw observations enter the attention computation only through this embedding. The construction of the embedding, whether from single observations, patches, or quantized values, is the subject of Subsection 3.3.

For a one-step forecast, we use the attention output at the last observed position, since the input window ends at $t-1$. The forecast can be written as

$$\hat{\boldsymbol{y}}_t = \sum_{i=1}^{L} \underbrace{a_i(\boldsymbol{y}_{t-L:t-1})}_{\text{data dependent}} v(\boldsymbol{y}_{t-i}), \qquad v(\boldsymbol{y}_{t-i}) = W_V^\top E(\boldsymbol{y}_{t-i}). \tag{5}$$

Then the weight on lag i is

$$a_i(\boldsymbol{y}_{t-L:t-1}) = \frac{\exp\left(q_t^\top k_{t-i}/\sqrt{d_k}\right)}{\sum_{j=1}^{L} \exp\left(q_t^\top k_{t-j}/\sqrt{d_k}\right)}, \qquad q_t = W_Q^\top E(\boldsymbol{y}_{t-1}), \quad k_{t-j} = W_K^\top E(\boldsymbol{y}_{t-j}). \tag{6}$$

Each $a_i(\boldsymbol{y}_{t-L:t-1})$ is nonnegative, and the weights sum to one across $i = 1, \ldots, L$ by the softmax construction. The query q_t is computed from the most recent available token, $E(\boldsymbol{y}_{t-1})$, and plays the role of the forecasting position at the end of the look-back window. The keys k_{t-j} are computed from the lagged tokens and summarize the candidate past dates. The score $q_t^\top k_{t-j}$ therefore measures how strongly the forecasting position attends to lag j. In this single-layer representation, q_t depends only on the last observed token $E(\boldsymbol{y}_{t-1})$. In a stacked transformer, the representation at that position has already mixed information from earlier tokens, so the query can depend on the wider history. Each attention weight also depends on the full window through the softmax normalization across all lags.

Equation (5) is a weighted average of transformed lags. The weights $a_i(\boldsymbol{y}_{t-L:t-1})$ play the same formal role as the coefficient matrices A_i in (1), but the analogy has two limits. In a VAR, the coefficient matrices are fixed after estimation and are applied to every window. In attention, the weights are functions of the current look-back window and are recomputed at each date t. The attention representation is also nonlinear. Nonlinearity enters through the value map $v(\cdot)$ and through the softmax that produces the weights.

In the running macro example, the attention weights in (5) may give different importance to past inflation or interest-rate episodes depending on the current state of the economy. A recent high-inflation window and a tranquil window can therefore induce different effective lag profiles from the same fitted model. This adaptivity is useful for forecasting, but the resulting weights are not fixed coefficients and should not be read as impulse responses.

The same operation also admits two useful econometric readings, as previewed in Figure 2: attention as a learned weighted estimator and attention as adaptive pooling. Firstly, for a single head, (6) has the form of a Nadaraya–Watson kernel smoother. The query plays the role of the evaluation point, the keys play the role of design points, and the values are the quantities being averaged. The similarity kernel is

$$K(q, k) = \exp(q^\top k/\sqrt{d_k}).$$

Attention differs from a classical smoother because similarity is not measured in the raw input space. The matrices W_Q and W_K learn the representation in which tokens are compared. The quantities being averaged are also learned through W_V. The second interpretation is adaptive pooling. Stacking (3) over the sequence gives

$$\text{Attention} = AV, \qquad A = \text{softmax}(QK^\top/\sqrt{d_k}), \qquad V = XW_V.$$

Here A is a row-stochastic $L \times L$ weight matrix. Each row of A gives the weights used at one token position, and the rows of V are the learned value vectors being pooled. The operation therefore combines information across tokens through weights that depend on the current input. This is

related to the pooling logic of factor models, but the analogy is limited. Dynamic factor models [5, 58] summarize a high-dimensional panel through a small number of common components with loadings fixed after estimation. Attention instead forms input-dependent weighted averages of value vectors. The resulting representation need not be low-dimensional. Multi-head attention repeats this adaptive pooling in parallel, and stacked layers apply it repeatedly.

Because (3) constructs an $L \times L$ matrix of scores, its computational cost scales quadratically with the context length. This becomes costly in forecasting applications, where long look-back windows are often used to capture seasonality, cycles, and other persistent dependence. An early wave of time-series transformers therefore focused on reducing this cost. Zhou et al. [70] retain only the most informative queries, reducing the cost to order $L \log L$, and decode the forecast horizon in a single pass. Wu et al. [64] embed a seasonal–trend decomposition and replace inner-product attention with an auto-correlation operator. Zhou et al. [71] move the computation to a frequency basis and obtain linear complexity. These models reported improvements over the unmodified transformer on standard long-horizon benchmarks, but the gains proved less settled than they first appeared.

Distribution shift is a separate problem, and it is addressed by a separate device. Kim et al. [26] propose reversible instance normalization (RevIN), which removes the location and scale of each window before modelling and restores them afterwards. It is a neural counterpart of the detrending, demeaning, and rescaling that classical practice applies before estimation. Later work models the shift rather than only removing it, feeding the discarded information back into the attention computation [37].

The limits of these early transformer forecasters were made explicit by Zeng et al. [69]. They showed that DLinear, essentially a single linear layer applied after seasonal–trend decomposition, matched or outperformed these architectures on the same benchmarks. Their diagnosis was that point-wise tokens carry limited information, autoregressive or iterated decoding can accumulate errors, and some benchmarks reward smooth extrapolation, where simple linear maps are already strong.

The lesson is that making attention cheaper or more elaborate is not enough if the object being attended to is poorly chosen. The transformer approach was therefore not abandoned. Instead, the literature reconsidered the unit over which attention is computed. That unit is the token, and it is the subject of the next subsection.

3.3 Tokenizing time series

Every architecture in this review operates on tokens, a representation inherited from natural language processing. The borrowing is worth making explicit, because it is both a source of these methods' power and a source of their characteristic difficulties. In language modelling, a token is the elementary unit consumed by the model. Since neural networks compute on vectors rather than symbols, raw text must first be segmented into discrete units. Modern systems typically use subword units, so a frequent word may be one token while a rare word is assembled from several pieces. Two steps should be kept distinct. Tokenization assigns a discrete index, which carries no meaning by itself. Embedding maps that index to a learned vector, and the usable representation resides in this vector space. A sentence therefore becomes a sequence of embeddings, stacked as $X \in \mathbb{R}^{L \times d}$. This array is the input to the attention operation in (3) (Figure 3).

For a real-valued series there is no canonical token. The earliest time-series transformers treated a single observation as a token. The DLinear critique exposed the weakness of this choice: point-wise tokens carry little local structure, and simple linear extrapolation can be hard to beat on smooth long-horizon benchmarks. Later work therefore reconsidered the unit over which attention is computed.

Nie et al. [43] propose PatchTST, which groups P contiguous observations into a single token. Each token then carries local shape rather than a single value, and the sequence shortens from

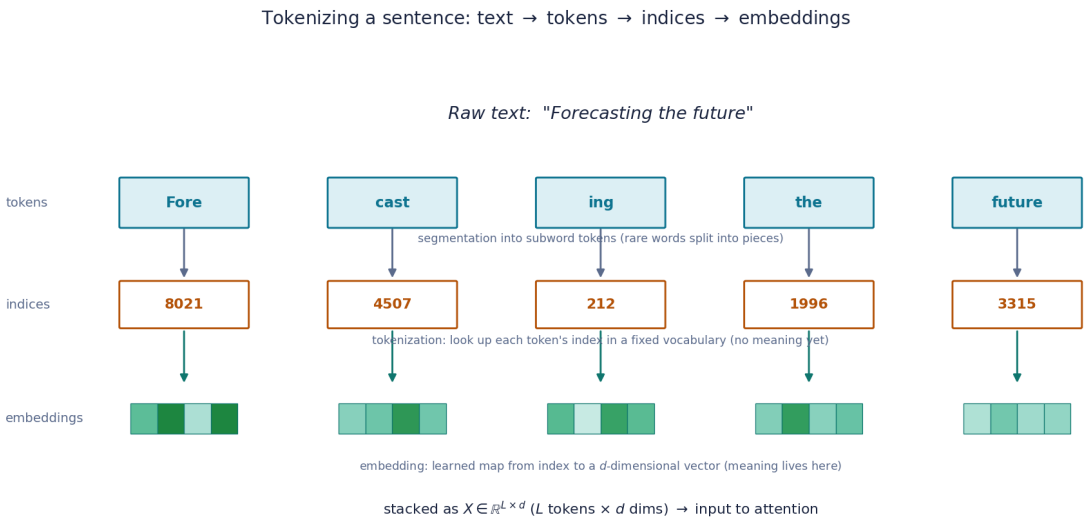


Figure 3: Tokenization in language modelling. A sentence is segmented into subword tokens, here "Forecasting" splitting into `Fore`, `cast`, and `ing`; each token is assigned an index in a fixed vocabulary, which carries no meaning, and each index is mapped by a learned embedding to a vector, in which meaning resides. The embeddings, stacked as $X \in \mathbb{R}^{L \times d}$, are the input to attention. Foundation models for time series replace the text tokens of this pipeline with quantized values or real-valued patches (Figure 4).

L to L/P. This reduces the cost of attention by a factor of roughly P^2. PatchTST also adopts channel independence, using a shared univariate model across series. This avoids fitting spurious cross-series correlations, but it also declines to model cross-series dependence directly.

Liu et al. [38] take the opposite view. In iTransformer, each variable's whole history is treated as a token, so attention runs across the cross-section rather than across time. Temporal structure is then handled mainly inside the feed-forward component. The contrast with PatchTST is useful: one design protects against spurious cross-series dependence by separating channels, while the other makes cross-series dependence the object of attention.

A third approach follows the language pipeline more literally. Ansari et al. [3] quantize each scaled observation into a fixed vocabulary of bins. The series becomes a sequence of discrete symbols, and an unmodified language-model architecture can be trained on it. The price is a resolution ceiling imposed by the binning. Quantized vocabularies and real-valued patches are the two constructions that organize much of the foundation-model literature, and they are contrasted in Figure 4.

The choice is not merely computational. The token fixes the objects over which attention is computed. For the macro panel, the model may compare individual quarterly observations, short patches of local dynamics, or entire variable histories. A patch token may encode a local inflation cycle. A variable token may ask how the inflation series as a whole relates to the policy-rate series. The token therefore fixes the economic object being compared. This is why tokenization provides the natural bridge from time-series transformers to foundation models.

Read against the three challenges, the transformer era is uneven. High dimensionality is handled mainly through implicit design choices. Channel independence sidesteps the cross-section by not modelling it directly. Cross-variable attention estimates dependence between series, but it provides no test of which series matter. Tokenization also shapes the effective dimension of the problem, since point-wise, patch-wise, and variable-wise tokens define different objects over which attention is computed. Nonstationarity is handled mostly through normalization. This can work well in practice, but it is a device rather than a model. It says nothing about unit

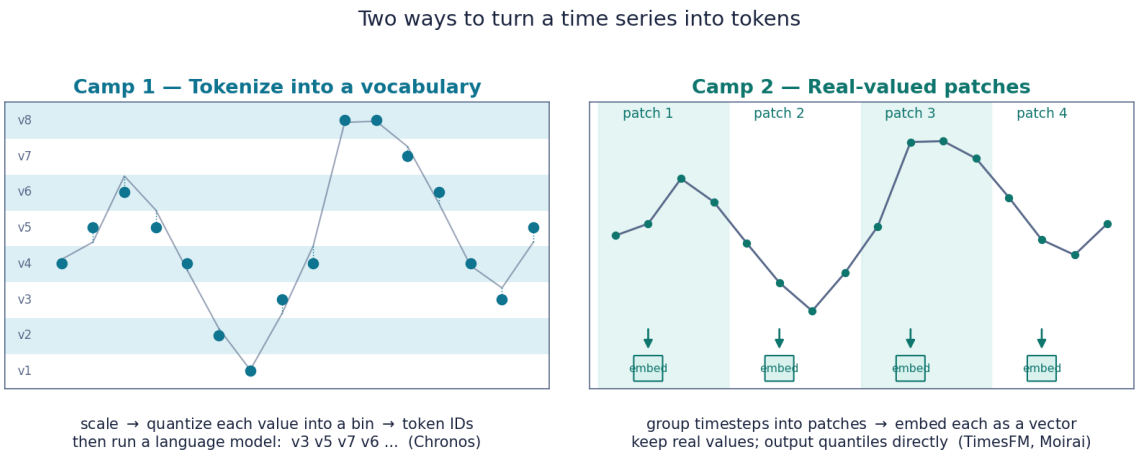


Figure 4: Two tokenization schemes for pretrained forecasters. Left: each scaled value is quantized into one of a fixed set of bins, so that the series becomes a sequence of vocabulary symbols that a language model continues (Chronos). Right: values are retained and contiguous timesteps are embedded as real-valued patches (TimesFM, Moirai). Quantization imposes a resolution ceiling; real-valued patches are compact and probabilistic by construction.

roots or cointegration, so a model fitted to normalized data has no notion of a long-run equilibrium. Later work models distribution shift rather than simply removing it, feeding the discarded information back into the attention computation [37]. The change-point and locally stationary methods of Section 2 have not yet been fully integrated into these architectures. Nonlinearity is where the main gain lies. Equation (3) gives a flexible operator whose weights adapt to the input, and it performs well when data are plentiful. The same pattern persists in the models below: clear gains against nonlinearity, ad hoc treatment of nonstationarity, and only implicit treatment of dimensionality.

4. Foundation models and zero-shot forecasting

Until recently, forecasting was usually organized around a per-data-set estimation workflow. For each application, the researcher selected a model, estimated its parameters on the available series, and then re-estimated the model as new data arrived. This workflow is difficult when the target series is short, because there is little data from which to estimate the model. Foundation models change the order of operations. The model is first pretrained on a large and heterogeneous collection of series. It is then applied to a new series by conditioning on that series' observed history, without re-estimating the model's parameters. In this sense, forecasting becomes less an exercise in estimating a new model for each data set and more an exercise in using a pretrained model with the right context.

The analogy with language modelling is direct. A language model is pretrained on many texts and then uses the prompt to generate a continuation. A time-series foundation model is pretrained on many series and then uses the observed history to generate a forecast. The time-series case, however, is harder in several ways. Series differ in scale, sampling frequency, seasonal pattern, and degree of nonstationarity. A model trained across many domains must therefore learn how to compare and use histories that may live on very different numerical scales.

In the macro example, zero-shot forecasting would mean applying a pretrained model to a new inflation-output-interest-rate panel without re-estimating its parameters. The econometric concern is whether the model is truly learning from the supplied history, or whether very similar macro series were already present during pretraining. This is the contamination problem. It is why post-cutoff evaluation is central: the evaluation series should come from a period that the

Table 1: Time-series foundation models discussed in the text, classified by tokenization, architecture, predictive output, and availability of weights. The two predominant tokenizations, quantized vocabularies and real-valued patches, are developed in Section 3.3.

Model	Tokenization	Architecture	Output	Weights
Chronos	quantized vocabulary	encoder–decoder	categorical	open
TimesFM	real-valued patches	decoder-only	quantiles	open
Moirai	real-valued patches	any-variate encoder	distributional	open
Lag-Llama	lagged covariates	decoder-only	distributional	open
TimeGPT	undisclosed	undisclosed	quantiles	closed
Time-MoE	point-wise real values	decoder-only MoE	point	open

pretraining corpus could not have included.

The operative mechanism is in-context learning. The observed history is given to the model as input, and the model adapts its forecast from that input alone. Its weights are not updated. This distinction is useful. Zero-shot forecasting refers to the setting in which the model forecasts a series that was not used for training. In-context learning refers to how the model adapts in that setting, by using the supplied history during a single forward pass. Context length is therefore central. A longer context lets the model condition on more of the target series. In multivariate settings, it can also include related series supplied alongside the target. The forecast then depends not only on what the model learned during pretraining, but also on how much relevant history is available at prediction time.

The tokenizations introduced in Section 3.3 also organize the pretrained models in Table 1. Chronos [3] represents the quantized-vocabulary approach. It reuses the language-model apparatus and treats forecasting as classification over value bins. This makes the connection to language modelling especially direct, but it also imposes a resolution ceiling on continuous forecasts. TimesFM [11] and Moirai [63] represent the real-valued-patch approach. TimesFM is a decoder-only, long-context model. Moirai is an any-variate, mixed-frequency model designed to handle different numbers of variables and different sampling frequencies. Real-valued patches have become a common design because they are compact, fast, and naturally suited to continuous-valued prediction. Other designs modify this template. Lag-Llama [47] encodes each date through lagged values. TimeGPT [16] is an early closed-weight commercial system. Time-MoE [51] pursues mixture-of-experts scaling. A recent development is the attachment of generative output heads, based on flow matching or diffusion, to pretrained backbones. This anticipates the convergence discussed in Section 5.

The empirical evidence is encouraging, but it should be read with care. On contamination-aware benchmarks such as GIFT-Eval [1], leading foundation models often outperform tuned statistical methods and many task-specific deep models in zero-shot mode (Table 2). Fine-tuning can improve performance, but the gains are often modest. This is itself informative, since it suggests that zero-shot forecasting should be tried before building a task-specific model. The main caveat is contamination. A pretrained model's zero-shot score is inflated if the evaluation series, or close relatives of it, appeared in the pretraining corpus. This leakage is hard to verify because pretraining data are often collected upstream and are not always fully disclosed. In econometric terms, it is a leakage problem with no close analogue in the usual per-data-set estimation setting. Benchmarks such as GIFT-Eval are therefore useful because they make contamination a central evaluation issue rather than an afterthought. Other limitations remain. Covariate support is uneven, irregular sampling is still difficult, and very long horizons remain challenging. In adversarial or strongly nonstationary domains, such as finance, the advantage of pretraining may disappear entirely. The operational rule is therefore unchanged: evaluate on genuinely held-out data, ideally post-cutoff and proprietary, and compare against strong classical baselines.

Table 2: Representative results from GIFT-Eval [1], computed from the public result files in the GIFT-Eval repository as accessed on 14 July 2026. Values are normalized weighted quantile losses (geometric mean across 97 benchmark tasks, relative to the seasonal-naive baseline), so seasonal naive equals one and lower is better. The upper block reports task-specific supervised baselines; the lower block reports zero-shot foundation models. TimesFM-2.5 denotes a later checkpoint of the model introduced by Das et al. [11], released without a separate publication. The leaderboard is updated over time, so these values should be read as a dated snapshot.

Model	Normalized WQL (↓)
Task-specific baselines	
Seasonal naive	1.000
Auto-ARIMA [22]	0.912
DeepAR [48]	0.853
DLinear [69]	0.846
N-BEATS [44]	0.816
Zero-shot foundation models	
Moirai-2.0 [35]	0.516
TimesFM-2.5 [11]	0.490
Chronos-2 [4]	0.485

Against the three challenges, foundation models address high dimensionality mainly through pretraining. Broad cross-corpus training lets the model pool information across many domains, and any-variate designs [63] allow the number of input series to vary. This differs from regularized or factor-based autoregressions, where parsimony is imposed through explicit and interpretable restrictions. Their treatment of nonstationarity is weaker. Scaling and normalization help models operate across heterogeneous series, but they do not replace the econometric apparatus of unit roots, cointegration, and error correction. Nonlinearity is inherited from the transformer backbone. The central innovation is therefore the replacement of in-sample estimation by cross-corpus pretraining, whose statistical properties remain only partly understood.

Recent theory outside time series interprets in-context learning as implicit Bayesian inference. In that view, the prompt updates a prior learned during pretraining, and risk falls as context length grows [42, 62, 65]. These results, however, usually assume independent examples drawn from stationary task mixtures. A forecasting context is different: it is a single dependent path, often nonstationary. Extending the theory to that setting remains an open problem.

5. Diffusion and generative forecasting

A useful forecast describes a range of possible futures, not just a single path. Decisions about inventory, capacity, and risk rely on quantiles and scenarios rather than a single point. Foundation models that output separate quantiles describe marginal uncertainty at each horizon, but these quantiles do not by themselves characterize the joint distribution across horizons and series. As a result, they cannot describe a coherent future scenario over the forecast horizon. Generative models address this limitation by sampling whole future paths (Figure 5). Among generative methods, diffusion has emerged as a leading framework for time-series forecasting, building on earlier successes in image and audio synthesis.

In the running example, a diffusion forecaster would not merely produce separate quantiles for future inflation, output growth, and the policy rate. It would sample joint future paths, so that a high-inflation scenario comes with a corresponding path for output and interest rates. The attraction is scenario coherence; the econometric question is whether those scenarios respect the structural and long-run relations thought to govern the macroeconomic system.

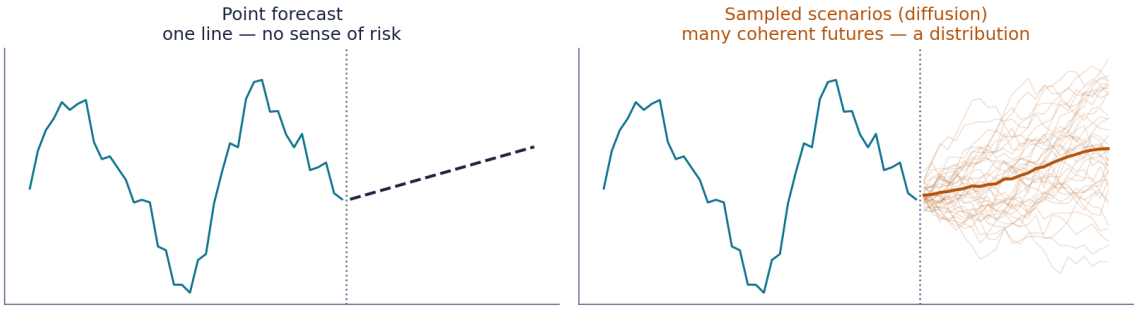


Figure 5: Why generative forecasting. A point forecast is a single trajectory and conveys little information about uncertainty (left). A diffusion model samples many coherent future paths, which together represent the predictive distribution (right).

The contrast with Section 2 is direct. In a Gaussian VARMA, the h-step predictive distribution is available in closed form,

$$y_{t+h} \mid y_{1:t} \;\sim\; \mathcal{N}\big(\hat{y}_{t+h},\, \Sigma_h\big), \tag{7}$$

so future uncertainty is summarized by a forecast mean and a covariance matrix. Diffusion takes a different route:

$$y_{t+1:t+H} \mid y_{1:t} \;\sim\; p_\theta\big(\,\cdot \mid y_{1:t}\big), \tag{8}$$

and learns to sample directly from the predictive distribution. It therefore does not impose a parametric constraint on its shape. The resulting distribution may be multimodal, heavy-tailed, or asymmetric in ways that a Gaussian VARMA (7) cannot capture. In this sense, diffusion targets the same forecasting object as the classical model, but represents it more flexibly.

Density forecasting has been approached with several classes of generative models, including variational autoencoders, normalizing flows, and generative adversarial networks. Each learns to represent the predictive distribution by a different mechanism. We focus here on denoising diffusion, which has become a leading approach for time series forecasting and is the subject of the remainder of this section. Broader reviews of generative forecasting methods are available elsewhere [41, 67].

A key limitation should be noted at the start. Neither diffusion nor the foundation models of Section 4 enforce cointegration or long-run equilibrium by construction. Where such structure must hold, the vector error-correction model in (2) remains the appropriate benchmark.

5.1 Denoising diffusion: mechanism and foundations

To fix ideas, we begin with the vanilla diffusion mechanism and introduce forecasting-specific conditioning only in Section 5.2.

Denoising diffusion combines a fixed forward process that gradually corrupts a target sample with a learned reverse process that reconstructs it (Figure 6). The basic idea goes back to Sohl-Dickstein et al. [53], who formulated generation as the finite-time reversal of a Markov chain that diffuses data into a tractable prior. It became practically useful with the denoising diffusion probabilistic models of Ho et al. [21]. Throughout this subsection, let $x^{(0)}$ denote the target sample to be generated, with the superscript indexing diffusion step rather than calendar time. In the forward process, each diffusion step shrinks the current state and adds Gaussian noise according to a prescribed variance schedule:

$$x^{(n)} = \sqrt{\alpha_n}\, x^{(n-1)} + \sqrt{\beta_n}\, \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, I).$$

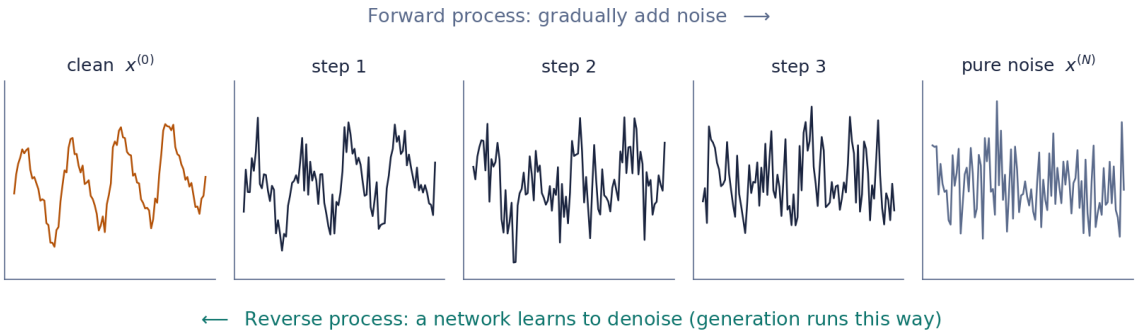


Figure 6: The diffusion mechanism. A fixed forward process gradually adds noise to the series from left to right. A learned reverse process removes that noise from right to left. Generation starts from pure noise and ends in a sample. Conditioning on the observed history turns this generative process into forecasting.

Here $\{\beta_n\}_{n=1}^N$ is a prescribed sequence that controls the amount of noise added at each diffusion step, and $\alpha_n = 1 - \beta_n$. Writing $\bar{\alpha}_n = \prod_{s=1}^n \alpha_s$, iterating this recursion yields the closed form

$$x^{(n)} = \sqrt{\bar{\alpha}_n}\, x^{(0)} + \sqrt{1 - \bar{\alpha}_n}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{9}$$

This representation allows training at any diffusion step without simulating the full chain. As n grows, the distribution approaches the standard normal prior from which generation begins.

For sufficiently small per-step variance, the reverse transition can be approximated by a Gaussian, with a mean predicted by the network. After the noise-prediction reparameterization of Ho et al. [21], the variational bound yields a timestep-weighted squared-error objective. In practice, one often optimizes the simplified objective

$$\mathbb{E}_{n,\, x^{(0)},\, \epsilon} \big\| \epsilon - \epsilon_\theta\big(x^{(n)}, n\big) \big\|^2, \tag{10}$$

where the expectation is taken over a diffusion step n drawn uniformly from $\{1, \ldots, N\}$, a clean sample $x^{(0)}$, and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$. This objective asks the model to recover the noise injected in the forward process. Under this parameterization, the reverse update draws $x^{(n-1)}$ from a Gaussian with mean

$$\mu_\theta\big(x^{(n)}, n\big) = \frac{1}{\sqrt{\alpha_n}} \left(x^{(n)} - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}}\, \epsilon_\theta\big(x^{(n)}, n\big) \right), \tag{11}$$

so that generation proceeds by repeatedly applying the learned denoiser from level N back to the data.

Up to a known scale factor, predicting the noise is equivalent to estimating the score, that is, the gradient of the log-density of the noised data. This is the perspective developed by Song and Ermon [55], who learn that gradient directly at a range of noise levels and generate samples by annealed Langevin dynamics. In that formulation, generation repeatedly follows the learned score field back toward high-density regions of the data distribution.

These two views, the variational one of Ho et al. [21] and the score-based one of Song and Ermon [55], were unified by Song et al. [56] within a single stochastic differential equation (SDE) framework. In this formulation, the forward SDE gradually injects noise, while the reverse-time SDE, driven by the learned score, defines the generative process. The same framework also admits a deterministic probability-flow ordinary differential equation (ODE) with the same marginals. In continuous time, this ODE plays the same role as the deterministic denoising diffusion implicit model (DDIM) sampler of Song et al. [54].

A main practical drawback of classical diffusion sampling is that it requires tens to hundreds of denoising steps. The probability-flow ODE and DDIM already reduce this cost by replacing

stochastic reverse sampling with more efficient deterministic trajectories. Subsequent developments push this idea further, including flow matching [33], which replaces the noising chain with a learned velocity field along a prescribed probability path, and, more recently, consistency models [57] and rectified flow [36]. These methods can reduce generation to a handful of function evaluations, making deployment far cheaper.

The forward process also gives formal content to the analogy drawn in Section 2, though the comparison should be interpreted with care. A single forward step follows the recursion $x^{(n)} = \sqrt{\alpha_n}\, x^{(n-1)} + \sqrt{\beta_n}\, \epsilon_n$, whose unrolling is exactly the closed form (9). The noised state is therefore a weighted accumulation of Gaussian terms $\{\epsilon_s\}_{s \le n}$, with the same moving-average algebra as the Wold representation, in which the VARMA outcome accumulates past innovations. The resemblance is formal rather than substantive. VARMA innovations are disturbances of the data generating process, indexed by calendar time and carrying economic meaning. Diffusion terms, by contrast, are artificial noise injected by the modeller. They are indexed by noise level rather than calendar time and are used only to define the generative mechanism. What the two constructions share is a linear Gaussian recursion and the tractability it affords. In the VARMA, that recursion yields a closed-form predictive density. In diffusion, it yields the closed form (9), which makes training feasible at arbitrary noise levels. The key departure is that diffusion learns to invert this accumulation, so the reverse map is nonlinear.

5.2 Conditional diffusion for time-series forecasting

To adapt vanilla diffusion to forecasting, let the target sample be the future segment

$$x^{(0)} := \boldsymbol{y}_{t+1:t+H}.$$

The forward noising process is unchanged. It perturbs only this future segment, while the observed history $\boldsymbol{y}_{1:t}$ enters separately as conditioning information.

In the unconditional setting of Section 5.1, the reverse chain learns transitions of the form $p_\theta(x^{(n-1)} \mid x^{(n)})$. For forecasting, these become conditional transitions

$$p_\theta\big(x^{(n-1)} \mid x^{(n)}, \boldsymbol{y}_{1:t}\big).$$

Equivalently, one may write the model in terms of a conditional denoiser $\epsilon_\theta(x^{(n)}, n \mid \boldsymbol{y}_{1:t})$. Running the reverse chain from the Gaussian prior therefore produces a sample from the conditional predictive distribution

$$p_\theta\big(\boldsymbol{y}_{t+1:t+H} \mid \boldsymbol{y}_{1:t}\big),$$

which is the forecasting object introduced in (8). In this sense, conditional diffusion forecasting preserves the vanilla generative mechanism and changes only the information supplied to the reverse map.

With this conditional formulation in place, the rest of the subsection organizes diffusion forecasters into four groups. Because time series have distinctive structure, including temporal dependence, seasonality, trend, multiscale patterns, and cross-variable interactions, plain diffusion algorithms are often modified or re-engineered to fit these features. The first group is organized by how the forecast horizon is generated. Autoregressive models such as TimeGrad and ScoreGrad produce one multivariate observation at a time, whereas whole-segment models such as CSDI and SSSD generate the entire future segment jointly. The second group focuses on how conditioning enters. TimeDiff strengthens the conditioning signal during training, while TSDiff trains an unconditional model and introduces the observed history only at sampling through self-guidance. The third group adds time-series structure to the diffusion process itself. Multiscale and decomposition models connect denoising to temporal aggregation, seasonal–trend structure, or frequency-based components. The final group changes the role of diffusion more substantially, using it either to support denoising and latent representation learning in short noisy series or to model dispersion around a transformer point forecast. Table 3 summarizes the models discussed below.

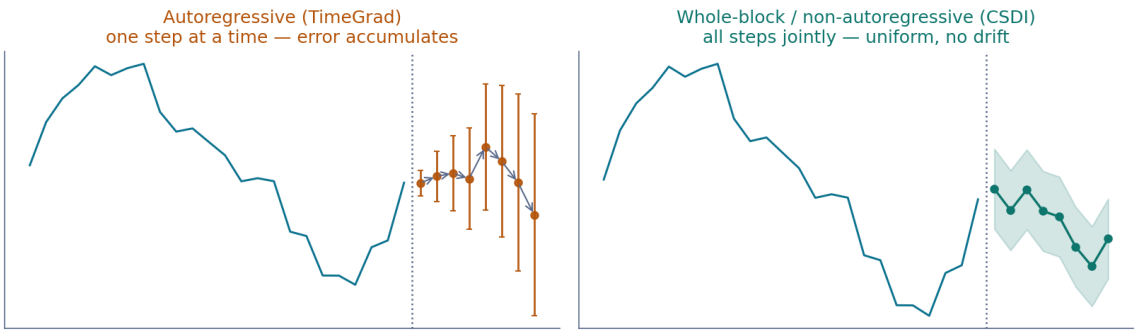


Figure 7: Autoregressive versus whole-segment generation. TimeGrad generates one date at a time, so the predictive band widens and error compounds along the horizon (left); CSDI and SSSD generate the whole horizon jointly, avoiding recursive feedback of previously sampled values (right).

Autoregressive versus whole-segment generation. The future horizon can be produced in two ways. A model may generate one date at a time, feeding each sampled value back as an input for the next. Or it may generate the whole segment $\boldsymbol{y}_{t+1:t+H}$ at once, in a single pass of the reverse chain. The distinction is not new to diffusion. It is the generative version of an older choice in forecasting, between iterated and direct multi-step prediction [40]. Iteration exploits the recursive structure of the process. It is efficient when the one-step model is correct, but error compounds along the horizon, because each step conditions on values the model has itself generated. Whole-segment generation avoids that accumulation. It is more robust to a misspecified one-step model, but it must capture dependence across the entire horizon in a single map.

Rasul et al. [46] propose TimeGrad, the first diffusion model for multivariate forecasting. It summarizes the observed history in the hidden state of a recurrent network. That state is passed to the denoiser as conditioning information, and the model generates one future date at a time. The samples are coherent across series. But the design inherits both costs of iteration. A full denoising loop is run at every forecast date and error accumulates along the horizon (Figure 7, left). Yan et al. [66] introduce ScoreGrad, the continuous-time counterpart of this autoregressive design. The Recurrent Neural Network (RNN) still summarizes the history and the forecast is still generated one date at a time, but the conditional distribution at each date is modeled by a score-based stochastic differential equation. This replaces the fixed denoising loop with numerical solvers for the reverse-time SDE, while leaving the autoregressive horizon factorization unchanged. ScoreGrad therefore inherits the same possibility of horizon-wise error accumulation.

A different line of work begins with CSDI [59], a conditional score-based diffusion model originally developed for imputation. The key observation is that forecasting can be viewed as imputation in which the missing segment lies at the end of the window. CSDI therefore masks the future and generates the entire horizon jointly, conditioned on the observed past. This removes autoregressive drift and allows the same model to handle ordinary missing-data imputation (Figure 7, right). Alcaraz and Strodthoff [2] propose SSSD, which keeps this whole-segment, mask-based design and changes only the backbone. Attention is replaced by the structured state-space (S4) layers of Gu et al. [19]. These capture long-range dependence at near-linear cost and scale more naturally to long sequences.

Conditioning refinements. A second set of models changes not the horizon factorization, but the way conditioning is supplied.

Shen and Kwok [49] propose TimeDiff, a non-autoregressive forecaster that strengthens the conditioning signal used by the denoiser. Two devices do the work. Future mixup blends ground-

truth future values into the conditioning signal during training, so that the denoiser sees future-compatible structure beyond what the history alone provides. When forecasting, however, these ground-truth values are unavailable and the condition is replaced by the history-based representation. Autoregressive initialization then starts the reverse chain from a cheap linear forecast rather than from pure noise. The model therefore refines an informed trajectory instead of building the future segment from scratch. Both devices are forecasting-specific design choices that improve empirical performance.

Kollovieh et al. [28] take the opposite route and introduce TSDiff, an unconditional diffusion model for time-series windows. Unlike conditional forecasters, TSDiff is trained with no forecasting condition supplied during training. The observed history enters only at sampling time through self-guidance. The motivation is the decomposition

$$\nabla_x \log p\big(x \mid y_{1:t}\big) = \nabla_x \log p\big(x\big) + \nabla_x \log p\big(y_{1:t} \mid x\big), \tag{12}$$

which shows that an unconditional score can be converted into a conditional one by adding a guidance correction. TSDiff approximates this correction at each denoising step by first using the denoiser to estimate the clean window implied by the current noisy sample. It then compares the history part of that estimate with the observed history and nudges the reverse update toward better agreement. The resulting sampler produces realistic windows whose past matches the observed history, and the future part of each window is read as the forecast.

Guidance buys generality: one trained model can be steered toward forecasting, forecast refinement, or synthetic generation without retraining. It costs exactness, since the correction in (12) is approximated rather than computed, so the guided sampler is not guaranteed to draw from the true conditional predictive law. Conditioning supplied during training targets that law more directly, but ties the model to the task it was trained for.

Multiscale and decomposition. A further group builds temporal structure into diffusion by linking the noise ladder to coarse-to-fine representations of the series (Figure 8). The intuition is that high noise levels preserve only broad low-frequency structure, while lower noise levels recover local fluctuations and high-frequency detail. This makes the reverse process resemble a coarse-to-fine reconstruction of the time series.

Fan et al. [14] introduce MG-TSD, which derives coarse-grained versions of the series by temporal aggregation and uses them as intermediate guidance targets during diffusion training. The motivation is that both diffusion noising and temporal aggregation remove fine-scale variation, leaving smoother low-frequency structure. The reverse process is therefore encouraged to recover broad temporal patterns before refining high-frequency detail. Shen et al. [50] propose mr-Diff, which makes the hierarchy explicit by generating a seasonal–trend decomposition from coarse to fine, with each finer stage conditioned on the coarser one. Yuan and Qiao [68] introduce Diffusion-TS and place the decomposition inside the denoiser itself: each denoising estimate is split into interpretable trend and Fourier-based seasonal components. Together, these methods make diffusion forecasting less purely generic by tying denoising to temporal scale and decomposition.

Distinct designs. The preceding models differ mainly in how they condition or guide a diffusion sampler whose target is the future segment. The following two models change the role of diffusion itself.

Li et al. [31] propose D^3VAE for short and noisy time series, where whole-segment diffusion models can be difficult to fit with limited data. In this model, diffusion is used less as a standalone generator and more as a denoising and augmentation device. A coupled forward process perturbs the history and the future together, creating noisy versions of a small training sample. A bidirectional variational autoencoder then learns to invert this process. The objective combines a denoising term, which removes both injected and intrinsic noise, with a disentanglement penalty that separates latent factors.

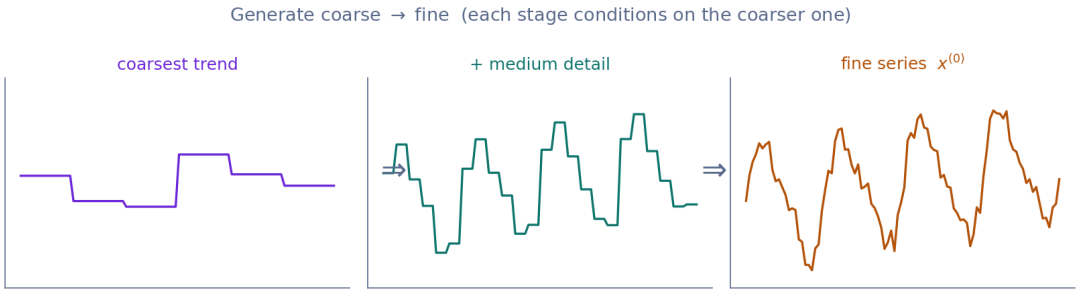


Figure 8: Multiscale structure in diffusion. MG-TSD uses coarse-grained series as guidance targets at intermediate diffusion levels, whereas mr-Diff explicitly generates the forecast from coarse trends to finer details.

Table 3: Diffusion forecasters, classified by generation scheme, core design, and conditioning mechanism.

Model	Generation	Core design	Conditioning
TimeGrad	autoregressive	RNN	recurrent state
ScoreGrad	autoregressive	RNN + SDE	recurrent state
CSDI	whole-segment	attention	masked observations
SSSD	whole-segment	state-space (S4)	masked observations
TimeDiff	whole-segment	convolutional	future mixup + AR init
TSDiff	unconditional	state-space (S4)	sampling-time self-guidance
MG-TSD	autoregressive	RNN (multi-granularity)	coarse-grained guidance
mr-Diff	whole-segment	seasonal–trend	coarse-to-fine stages
Diffusion-TS	whole-segment	decomposition attn.	guidance
D^3VAE	whole-segment	diffusion + BVAE	latent
TMDM	whole-segment	transformer + diff.	conditional mean

Li et al. [32] propose TMDM, which combines a transformer point forecaster with a diffusion model for residual uncertainty. The transformer first produces a conditional mean forecast. Rather than diffusing toward a standard normal prior, the forward process diffuses toward a Gaussian centered at that forecast, $\mathcal{N}(\hat{y}, I)$, and the same transformer output also modulates the reverse chain. Diffusion therefore models plausible deviations around a strong conditional mean, rather than generating the future segment from scratch. This design connects the point-forecasting models of Sections 3–4 with the generative forecasting methods of this section.

The diffusion literature therefore answers the three challenges unevenly. It is strongest where the problem is joint uncertainty, less settled where the issue is nonstationary structure, and broadly flexible for nonlinear and non-Gaussian predictive shapes.

Diffusion is most distinctively valuable for high dimensionality. By learning a joint predictive law over horizons and series, diffusion produces coherent scenario paths rather than separate marginal quantiles. This is precisely what quantile heads alone cannot provide.

For nonstationarity, the picture is less settled. For nonstationarity, the picture is less settled. Recent work has begun to incorporate time-series characteristics into diffusion design. For example, Lee et al. [29] select the noise schedule using autocorrelation-based measures of nonstationarity. This remains an adaptive scheduling device rather than a formal treatment of unit roots or cointegration. The vector error-correction form (2) therefore remains the appropriate guardrail when a long-run equilibrium must hold.

Nonlinearity and non-Gaussianity are handled more naturally. Diffusion imposes no parametric form on the predictive distribution, so it can represent multimodal, heavy-tailed, and asymmetric forecast laws beyond the Gaussian VARMA. The broader trajectory is to place

Family	High dimensionality	Nonstationarity	Nonlinearity	Inference
VAR / VECM	factor, shrinkage, sparse priors (explicit)	unit roots, cointegration (explicit)	linear, fixed coefficients	full distribution theory
Transformers	channel independence / cross-variable attention	instance normalization (RevIN)	data-dependent attention	none standard
Foundation models	cross-corpus pretraining; any-variate	scaling across regimes (implicit)	inherited from backbone	none standard
Diffusion	joint predictive law; no explicit dimension reduction	limited explicit treatment	non-Gaussian, multi-modal	sampling-based only

Table 4: How each method family treats the three challenges, and what inferential apparatus it provides. "Explicit" denotes an interpretable, testable restriction; "implicit" a data-driven mechanism without such guarantees.

such generative heads on top of pretrained backbones, combining transformer representations, pretraining-based generalization, and diffusion-based scenario uncertainty.

6. Synthesis and outlook

Table 4 summarizes the argument. Read across the challenge columns, no family yet meets high dimensionality, nonstationarity, and nonlinearity with the completeness applied work requires, and the pattern is uneven. Progress on nonlinearity and non-Gaussian predictive shape has been rapid. High dimensionality is handled implicitly, through pooling, channel choices, pretraining, or joint sampling, rather than through the interpretable restrictions of factor and shrinkage methods. Nonstationarity is met with normalization or adaptation, not with the unit-root and cointegration apparatus that economic series often require. The structural and inferential machinery of the classical column, orthogonalized shocks, testable restrictions, and equilibrium-restoring terms, has no clear counterpart further right.

These advances make the modern architectures flexible representations of the conditional law of the future given the past. Flexibility is not by itself an econometric standard. For a method to inform policy, risk management, or scientific interpretation, its use must also be governed by credible evaluation, calibrated uncertainty, explicit identification, and a clear account of the nonstationarity it is meant to handle.

Evaluation should mimic the information set available at the forecast origin. This rules out random train–test splits for dependent data, in favour of rolling-origin or expanding-window designs. For pretrained models a further concern is contamination. A zero-shot score is uninformative if the target series, or close variants of it, entered the pretraining corpus, so the strongest evidence comes from post-cutoff or proprietary data, or from benchmarks that separate the pretraining and evaluation corpora. Modern models should be compared against strong classical baselines, including seasonal naive forecasts, ARIMA, VAR or Bayesian VAR, and a VECM where cointegration matters.

A forecast is often useful only with a statement of uncertainty. Classical Gaussian VARs supply this through an explicit sampling theory. Modern neural forecasters usually do not. Quantile heads, diffusion samplers, and ensembles produce empirical predictive distributions, but these should not be read as calibrated without a check. Calibration can be assessed through interval coverage, probability integral transform diagnostics, and horizon-by-horizon reliability. Where

distributional assumptions are weak, conformal prediction offers model-agnostic finite-sample coverage under exchangeability or approximate-stability conditions. The operative question is not whether a model gives sharp intervals, but whether those intervals have their advertised coverage under the instability faced in the application.

Forecast accuracy does not identify structural effects. The reduced-form VAR forecasts well, but its innovations are correlated, so an impulse response becomes economically meaningful only after identifying restrictions, such as a recursive ordering, sign restrictions, external instruments, or long-run restrictions. The flexible methods inherit this problem and add nothing to its solution. Attention weights, learned kernels, and diffusion trajectories are functions of the fitted predictive system, not structural shocks. A data-dependent weight is not an identified parameter, and its sampling distribution is unknown. Structural use therefore requires two steps that accuracy cannot supply. First, define the estimand, for example a nonlinear analogue of an impulse response or a counterfactual policy path. Second, supply assumptions that connect it to the data, by imposing econometric restrictions on the flexible model or by combining it with external identification. In the macro panel, if the question is the effect of a monetary tightening, the model must be linked to an identified policy shock. A diffusion path in which the interest rate rises does not by itself identify one. Absent such structure, modern forecasters are predictive devices rather than structural models.

Nonstationarity is not a single problem. Series may carry stochastic or deterministic trends, structural breaks, evolving volatility, or gradual parameter drift, and classical econometrics distinguishes these cases because each calls for a different response. Modern systems tend to address them together, through scaling, normalization, long context windows, or pretraining across regimes. These devices improve robustness but do not replace an explicit account of the instability, and differencing or normalizing away low-frequency movement can destroy long-run equilibrium information when series are cointegrated.

Taken together, the move from vector autoregressions to transformers, foundation models, and diffusion is best understood as an expansion of the forecasting class, not a replacement of econometric structure. Self-attention keeps the VAR's weighted sum of past values, but lets the weights depend on the input and adds nonlinearity. Foundation models replace per-data-set estimation with pretraining across many series and zero-shot adaptation to new ones. Diffusion replaces a single parametric predictive density with samples of coherent future paths. Each step adds flexibility, scale, and richer predictive distributions. None of them, by itself, supplies the explicit restrictions, identification, and inferential discipline that structural interpretation and policy analysis require. In particular, the burden of dimensionality is partly shifted from estimating many parameters on one data set to learning shared representations from large and heterogeneous corpora. It is striking that modern machine-learning methods address the dimensionality problem in such a natural way: rather than specifying low-dimensional restrictions by hand, they learn reusable representations through pooling, attention, and pretraining across many related series.

The task ahead is to combine the predictive strength of modern architectures with the structural and inferential tools of econometrics. Developing reliable inference and interpretability for these models is a promising direction, because it would clarify when their predictive distributions are calibrated, when learned restrictions can be interpreted econometrically, and how far their black-box representations can support policy or scientific claims rather than forecasts alone. Determining when implicit restrictions, learned through pretraining and pooling, outperform explicit econometric structure under dependence, heterogeneity, and instability remains, in our view, the central open problem this literature poses.

References

[1] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

[2] Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *Transactions on Machine Learning Research (TMLR)*, 2023.

[3] Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Bernie Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.

[4] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.

[5] Jushan Bai and Serena Ng. Determining the number of factors in approximate factor models. *Econometrica*, 70(1):191–221, 2002.

[6] Marta Bańbura, Domenico Giannone, and Lucrezia Reichlin. Large Bayesian vector auto regressions. *Journal of Applied Econometrics*, 25(1):71–92, 2010.

[7] Sumanta Basu and George Michailidis. Regularized estimation in sparse high-dimensional time series models. *The Annals of Statistics*, 43(4):1535–1567, 2015.

[8] Olivier Jean Blanchard and Danny Quah. The dynamic effects of aggregate demand and supply disturbances. *The American Economic Review*, 79(4):655–673, 1989.

[9] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza Ramirez, Max Mergenthaler Canseco, and Artur Dubrawski. NHITS: Neural hierarchical interpolation for time series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(6): 6989–6997, 2023.

[10] Rainer Dahlhaus. Fitting time series models to nonstationary processes. *The Annals of Statistics*, 25(1):1–37, 1997.

[11] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *International Conference on Machine Learning (ICML)*, 2024.

[12] David A. Dickey and Wayne A. Fuller. Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366):427–431, 1979.

[13] Robert F. Engle and C. W. J. Granger. Co-integration and error correction: Representation, estimation, and testing. *Econometrica*, 55(2):251–276, 1987.

[14] Xinyao Fan, Yueying Wu, Chang Xu, Yuhao Huang, Weiqing Liu, and Jiang Bian. MG-TSD: Multi-granularity time series diffusion models with guided learning process. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Piotr Fryzlewicz. Wild binary segmentation for multiple change-point detection. *The Annals of Statistics*, 42(6):2243–2281, 2014.

[16] Azul Garza, Cristian Challu, and Max Mergenthaler-Canseco. TimeGPT-1. *arXiv preprint arXiv:2310.03589*, 2023.

[17] C. W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969.

[18] C.W.J. Granger and P. Newbold. Spurious regressions in econometrics. *Journal of Econometrics*, 2(2):111–120, 1974. ISSN 0304-4076.

[19] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

[20] James D. Hamilton. *Time Series Analysis*. Princeton University Press, 1994.

[21] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] Rob J. Hyndman and Yeasmin Khandakar. Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3):1–22, 2008.

[23] Søren Johansen. Estimation and hypothesis testing of cointegration vectors in gaussian vector autoregressive models. *Econometrica*, 59(6):1551–1580, 1991.

[24] Søren Johansen. *Likelihood-Based Inference in Cointegrated Vector Autoregressive Models*. Oxford University Press, 1995.

[25] Lutz Kilian and Helmut Lütkepohl. *Structural Vector Autoregressive Analysis*. Cambridge University Press, 2017.

[26] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*, 2022.

[27] Anders Bredahl Kock and Laurent Callot. Oracle inequalities for high dimensional vector autoregressions. *Journal of Econometrics*, 186(2):325–344, 2015.

[28] Marcel Kollovieh, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[29] Seunghan Lee, Kibok Lee, and Taeyoung Park. Ant: Adaptive noise schedule for time series diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[30] Jie Li, Paul Fearnhead, Piotr Fryzlewicz, and Tengyao Wang. Automatic change-point detection in time series via deep learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(2):273–285, 2024.

[31] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. Generative time series forecasting with diffusion, denoise, and disentanglement. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[32] Yuxin Li, Wenchao Chen, Xinyue Hu, Bo Chen, Baolin Sun, and Mingyuan Zhou. Transformer-modulated diffusion models for probabilistic multivariate time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

[33] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[34] Robert B. Litterman. Forecasting with Bayesian vector autoregressions—five years of experience. *Journal of Business & Economic Statistics*, 4(1):25–38, 1986.

[35] Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*, 2025.

[36] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

[37] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Advances in Neural Information Processing Systems*, 2022.

[38] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

[39] Helmut Lütkepohl. *New Introduction to Multiple Time Series Analysis*. Springer, 2005.

[40] Massimiliano Marcellino, James H. Stock, and Mark W. Watson. A comparison of direct and iterated multistep AR methods for forecasting macroeconomic time series. *Journal of Econometrics*, 135(1–2):499–526, 2006.

[41] Caspar Meijer and Lydia Y. Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.

[42] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations (ICLR)*, 2022.

[43] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

[44] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.

[45] Peter C. B. Phillips and Pierre Perron. Testing for a unit root in time series regression. *Biometrika*, 75(2):335–346, 1988.

[46] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning (ICML)*, 2021.

[47] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, Marin Biloš, Sahil Garg, Anderson Schneider, Nicolas Chapados, Alexandre Drouin, Valentina Zantedeschi, Yuriy Nevmyvaka, and Irina Rish. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.

[48] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.

[49] Lifeng Shen and James T. Kwok. Non-autoregressive conditional diffusion models for time series prediction. In *International Conference on Machine Learning (ICML)*, 2023.

[50] Lifeng Shen, Weiyu Chen, and James T. Kwok. Multi-resolution diffusion models for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

[51] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-MoE: Billion-scale time series foundation models with mixture of experts. In *International Conference on Learning Representations (ICLR)*, 2025.

[52] Christopher A. Sims. Macroeconomics and reality. *Econometrica*, 48(1):1–48, 1980.

[53] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 2256–2265, 2015.

[54] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[55] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[56] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[57] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[58] James H. Stock and Mark W. Watson. Forecasting using principal components from a large number of predictors. *Journal of the American Statistical Association*, 97(460):1167–1179, 2002.

[59] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[61] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pages 6778–6786, 2023.

[62] Noam Wies, Yoav Levine, and Amnon Shashua. The learnability of in-context learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[63] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *International Conference on Machine Learning (ICML)*, 2024.

[64] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[65] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations (ICLR)*, 2022.

[66] Tijin Yan, Hongwei Zhang, Tong Zhou, Yufeng Zhan, and Yuanqing Xia. ScoreGrad: Multivariate probabilistic time series forecasting with continuous energy-based generative models. *arXiv preprint arXiv:2106.10121*, 2021.

[67] Yiyuan Yang, Ming Jin, Haomin Wen, Chaoli Zhang, Yuxuan Liang, Lintao Ma, Yi Wang, Chenghao Liu, Bin Yang, Zenglin Xu, Jiang Bian, Shirui Pan, and Qingsong Wen. A survey on diffusion models for time series and spatio-temporal data. *arXiv preprint arXiv:2404.18886*, 2024.

[68] Xinyu Yuan and Yan Qiao. Diffusion-TS: Interpretable diffusion for general time series generation. In *International Conference on Learning Representations (ICLR)*, 2024.

[69] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[70] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[71] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning (ICML)*, 2022.